CHAPTER 14

Design A Search Autocomplete System

Return the top-5 suggestions within milliseconds as the user types each character (typeahead).

Jump to: Problem statement | Clarifying questions to ask | Requirements | Back-of-the-envelope estimation | API / data model | High-level design | Deep dive | Trade-offs and alternatives | Scaling, reliability, failure modes | Likely follow-up questions | Common mistakes | Interview answer script | 60-second recap / practice checklist | References and further reading

1. Problem statement

Design a search autocomplete (typeahead) system: when a user types a prefix such as `dinner`, return the top 5 most popular completions (e.g. `dinner ideas`, `dinner recipes`). It must respond in tens of milliseconds, since it fires on every keystroke.

Why interviewers ask it: A data-structure-meets-systems problem: tries, top-k caching, offline aggregation vs real-time updates, and sharding.

2. Clarifying questions to ask (and sensible assumptions)

Question	Assumed answer
Match at the beginning only?	Yes - prefix match.
How many suggestions?	5.
Which ones?	Top 5 by historical query frequency.
Spell check?	No.
Language?	English; lowercase alphabets.
Scale?	10 M DAU.

3. Requirements

Functional

- Given a prefix, return top-k completions ranked by popularity.
- Update popularity from new search queries.

Non-functional

- **Fast:** under 100 ms (target ~20 ms).
- Relevant, sorted by popularity.
- Scalable and highly available.
- Fresh enough (daily/hourly updates acceptable).

4. Back-of-the-envelope estimation

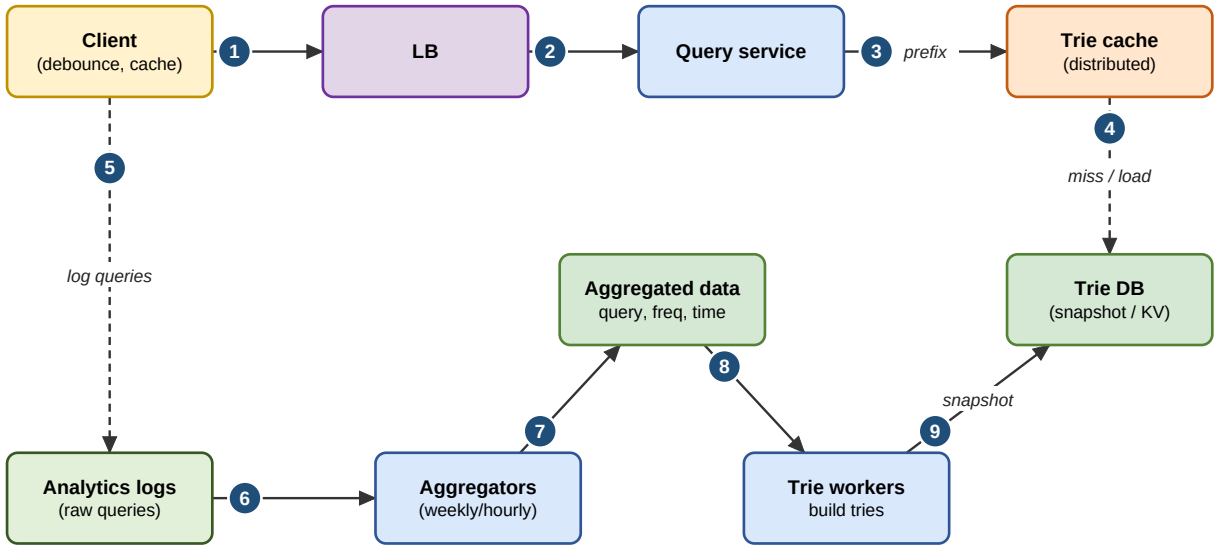
- 10 M DAU x 10 searches/day x ~20 chars typed = **about 24,000 QPS** (peak about 48 K). Each keystroke is a request.
- 20% of daily queries are new -> 10 M x 10 x 0.2 x 20 B = about 0.4 GB new data/day.
- Trie of the top queries easily fits in memory on a few machines when pruned.

5. API / data model

```
GET /autocomplete?q=<prefix>&limit=5  -> ["dinner ideas", "dinner recipes", ...]
```

6. High-level design

Data gathering (offline) and query (online) services



Reads hit a cached trie; writes never touch it directly - they are aggregated offline and rebuilt periodically.

Step	What happens
1	As the user types, the client (after a ~50 ms **debounce**, and checking its local cache) sends `GET /autocomplete?q=prefix`.
2	The load balancer routes to a stateless **query service**.
3	The query service looks up the prefix in the **trie cache**, where each node already stores its **top-5 completions**, so the answer is O(1) after reaching the node.
4	On a cache miss or at startup, the trie (or `prefix -> top-k` entries) is loaded from the **Trie DB** snapshot.
5	Separately, completed searches are **logged** (sampled) to analytics storage. This is the write path and never touches the trie directly.
6	**Aggregators** roll logs up (hourly or weekly) into `(query, frequency)` counts.
7	Aggregated data is stored with timestamps.
8	**Trie workers** rebuild the trie from the aggregates, with time decay and filtering of unsafe terms.
9	They publish a new snapshot to the Trie DB, which the cache picks up (versioned, with rollback).

7. Deep dive

Naive approach: SQL

- `SELECT * FROM queries WHERE query LIKE 'prefix%' ORDER BY freq DESC LIMIT 5` - fine for tiny data, far too slow at 24 K QPS.

Trie (prefix tree)

- Each node = character; path from root = prefix; node marks a word and its frequency. Look up the prefix node, then traverse the subtree to collect/rank completions.
- Cost: find prefix O(p); get all valid descendants O(c); sort O(c log c). Slow for long subtrees.

Optimisations

- **Limit prefix length** (e.g. 50 chars) -> prefix lookup becomes O(1).
- **Cache top-k at every node:** store the top 5 completions in each node -> answering is O(1) after reaching the node (trade space for time).
- Prune rarely used branches; store only queries above a frequency threshold.

Building and updating the trie

- Do **not** update on every query (too slow; top-k changes rarely for popular prefixes).
- **Offline:** log queries -> aggregate (weekly or hourly buckets) -> workers rebuild the trie -> store a snapshot in the Trie DB -> cache loads it.
- Real-time trending (news events) can be layered on with stream processing (a small, separate fast-updating trie merged at query time).
- Storage: serialise the trie in a document store (e.g. Mongo) or key-value store with `prefix -> top-k list` (each prefix as a key).

Query flow and client optimisations

- Browser cache results (`Cache-Control: private, max-age=3600`); **debounce** requests (wait ~50 ms after typing); AJAX requests; **data sampling** (log 1 of N queries).
- Use **keep-alive / WebSocket** connections to cut handshake overhead.

Scaling the storage

- Shard by the **first letter(s)** of the prefix; hot prefixes (`s`, `a`) are uneven -> shard map built from historical load (e.g. 's' alone on a shard, 'u-z' combined).
- Replicate shards for availability.

Delete / filter

- Filter hateful, violent, adult, or legally removed suggestions with a **filter layer** between the API and the trie cache. Remove asynchronously at the next rebuild.

Multi-language and trending

- Unicode tries per language/locale; store regional popularity separately for country-specific suggestions.

8. Trade-offs and alternatives

Decision	Option A / Option B	How to choose
Top-k at node vs compute on the fly	Store top-k per node (more memory, O(1) read) / traverse the subtree (small memory, slow)	Store top-k; memory is cheap relative to latency.
Update frequency	Real-time / hourly or weekly rebuild	Offline rebuild for stability; add a small real-time layer only for trending topics.
Storage	In-memory trie / KV store of prefix -> list	KV of prefixes is simpler to shard and replicate.

9. Scaling, reliability, failure modes

- Trie cache sharded and replicated; snapshot restore after crash.
- Most traffic is hot prefixes -> multi-layer cache (browser, CDN/edge, app cache).
- Version snapshots; roll out new trie gradually and roll back if quality drops.

10. Likely follow-up questions

- How do you show trending / personalised suggestions?
- How do you support multiple languages?
- How do you keep suggestions fresh in real time during breaking news?
- How would you shard and rebalance a skewed prefix distribution?

11. Common mistakes

- Updating the trie synchronously per search.
- Computing top-k at query time by walking the subtree.
- Uniform first-letter sharding ignoring skew.

12. Interview answer script (what to say, in order)

Follow the 4-step framework from chapter 04. Say it out loud; the quoted text is a model answer in your own voice.

Phase	Time	What to say
1. Understand + scope	4 min	"I'll assume prefix matching only, return the top 5 suggestions ranked by historical query frequency, English lowercase, no spell check, 10 M DAU. Since it fires on every keystroke, that is about 24,000 QPS and responses must come back in tens of milliseconds."
2. High-level design	10 min	"Two services. A data gathering path: log queries, aggregate counts offline, and build a trie. A query path: client sends the prefix, a query service reads the top-5 list from a trie cache. I keep reads and writes separate, so the serving path is read-only."
3. Deep dive	25 min	"A SQL LIKE query is too slow at this QPS, so I use a trie. Walking a trie and ranking the subtree is still slow, so I limit prefix length and store the top-k at every node, giving O(1) lookups at the cost of memory. I rebuild the trie offline from aggregated logs and publish snapshots, rather than updating it per search. For scale I shard by prefix with load-aware boundaries, replicate, cache in the browser and debounce on the client. A filter layer removes unsafe suggestions."
4. Wrap up	4 min	"Extensions: trending queries via a small real-time layer, personalisation, and multi-language support with per-locale tries. I'd monitor suggestion latency, cache hit rate and snapshot freshness."

13. 60-second recap / practice checklist

- Trie with cached top-k per node, built offline from aggregated query logs.
- Query path: client debounce -> LB -> query service -> trie cache -> trie DB.
- Shard by prefix, filter layer, cache everywhere.

14. References and further reading

Cross-checked against these sources (links verified when this PDF was built).

• GeeksforGeeks: Trie
• GeeksforGeeks: Design typeahead / autocomplete
• Elasticsearch completion suggester